

TD&TP10

ARBRE PARFAIT

- 1) Ecrire en C la déclaration de la structure d'un arbre Parfait (Tas).
- 2) Ecrire les fonctions suivantes :
 - initAP: initialise un arbre parfait.
 - estVideAP: teste si un arbre parfait est vide.
 - estPleinAP : teste si un arbre parfait est pleine.
 - estExistNoeudAP : teste l'existence d'un nœud donné dans un arbre parfait donné.
 - donneeAP : retourne la donnée associée à un nœud donné dans un arbre parfait donné.
 - pereNoeudAP : retourne le père d'un nœud donné dans un arbre parfait donné.
 - filsGaucheAP : retourne le fils gauche d'un nœud dans un arbre parfait donné.
 - filsDroitAP : retourne le fils droit d'un nœud dans un arbre parfait donné.
 - estFeuilleAP : teste si un nœud donné dans un arbre parfait donné est une feuille.
 - ParcoursPréfixeAP : effectue un parcours en profondeur dans l'ordre préfixe dans un arbre parfait donné
 - ParcoursInfixeAP : effectue un parcours en profondeur dans l'ordre infixé dans un arbre parfait donné
 - ParcoursPostfixeAP : effectue un parcours en profondeur dans l'ordre postfixé dans un arbre parfait donné
 - **ParcoursLargeur : effectue un parcours en largeur dans un arbre parfait donné.**

Chaque niveau de l'arbre parfait doit être affiché sur une ligne différente.

 - InserEltAP : insert un élément donné dans un arbre parfait donné (insertion à la première place libre trouvé).
 - ConstAP : construire un arbre parfait par ajouts successifs d'un certain nombre d'éléments.
- 3) Ecrire un programme principal pour tester les fonctions ci-dessus.

TAS-MAX (OU TAS-MIN)

Note : dans ce qui suit, on confond l'élément et sa priorité.

Ecrire en langage algorithmique ou en C les algorithmes ci-dessous :

- 1) Déterminer l'élément du sommet dans un tas donné.
- 2) Insérer un nouvel élément dans un tas donné (version itérative et **récurive**).
- 3) Supprimer l'élément maximal dans un tas donné (version itérative et **récurive**).
- 4) **Transformer un tableau composé de n éléments en un tas (version itérative et récurive).**
- 5) Ecrire un programme principal pour tester les fonctions ci-dessus.

APPLICATION : TRI PAR TAS

- 1) Trier un tableau composé n éléments à l'aide d'un tas.
- 2) Ecrire un programme principal pour tester le tri par tas.

EXERCICE 1

Ecrire une fonction booléenne **itérative (puis récurive)** qui vérifie si un tableau tab composé de n éléments peut être un tas.

➤ **Version 1** : raisonnement de la droite vers la gauche : on compare la donnée associée à chaque nœud avec la donnée associé à son nœud père.

➤ **Version 2** : raisonnement de la gauche vers la droite : on suppose que le tas est transformé en arbre parfait. On n'effectue aucune transformation par contre on raisonne d'une façon virtuelle.

EXERCICE 2

Implémenter d'une manière générique les algorithmes définis précédemment dans les arbres parfaits & tas.